

PROIECT CHAT JAVA

Matei Daniel Mîndruleanu – Universitatea din Bucuresti – BDTS

Descriere

În această aplicație, clienții se conectează la un server central și pot comunica în timp real prin trimiterea și recepționarea de mesaje. Fiecare client dispune de o interfață grafică (realizată cu Swing) care le permite să vizualizeze mesajele primite, să introducă mesaje noi și să le trimită. Serverul, pe rând, gestionează fiecare conexiune printr-un fir de execuție (ClientHandler) și transmite mesajele tuturor clienților conectați.

Componentele Proiectului

1. Clasa ChatMessage

Această clasă reprezintă elementul de bază al comunicației – un mesaj de chat. Ea implementează interfața *Serializable*, permițând transformarea obiectului într-un flux de biți, astfel încât să poată fi transmis prin rețea.

```
import java.io.Serializable;

public class ChatMessage implements Serializable {

    private String sender;

    private String message;

    public ChatMessage(String sender, String message) {

        this.sender = sender;

        this.message = message;

    }

    public String getSender() {

        return sender;

    }

    public String getMessage() {

        return message;

    }

}
```

2. Serverul (ChatServer)

Serverul reprezintă componenta centrală a arhitecturii client-server. El ascultă pe un port prestabilit (ex. 12345) și acceptă conexiuni de la clienți.

Responsabilități:

- Inițierea unui *ServerSocket* pentru a asculta conexiunile.
- Pentru fiecare client nou, se creează un obiect de tip *ClientHandler* (o clasă internă ce extinde *Thread*) care gestionează comunicarea cu clientul respectiv.
- Menținerea unei colecții sincronizate de clienți (*clientHandlers*) pentru a asigura că accesul la această listă este sigur în contextul concurenței.
- Implementarea unei metode sincronizate *broadcast* care trimite mesajele primite de la orice client către toți clienții conectați.

```
import java.io.*;
import java.net.*;
import java.util.*;

public class ChatServer {

    private static Set<ClientHandler> clientHandlers = Collections.synchronizedSet(new HashSet<>());

    public static void main(String[] args) {

        int port = 12345;

        try (ServerSocket serverSocket = new ServerSocket(port)) {

            System.out.println("Serverul a pornit pe portul " + port);

            while (true) {

                Socket socket = serverSocket.accept();

                System.out.println("Client nou conectat: " + socket);

                ClientHandler clientHandler = new ClientHandler(socket);

                clientHandlers.add(clientHandler);

                clientHandler.start();

            }

        } catch (IOException e) {

            e.printStackTrace();

        }

    }

    public static synchronized void broadcast(ChatMessage message) {

        for (ClientHandler client : clientHandlers) {

            client.sendMessage(message);

        }

    }

}
```

```

    }
}

public static void removeClient(ClientHandler clientHandler) {
    clientHandlers.remove(clientHandler);
}

static class ClientHandler extends Thread {
    private Socket socket;
    private ObjectOutputStream out;
    private ObjectInputStream in;
    private String clientName;
    public ClientHandler(Socket socket) {
        this.socket = socket;
        try {
            out = new ObjectOutputStream(socket.getOutputStream());
            in = new ObjectInputStream(socket.getInputStream());
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public void run() {
        try {
            while (true) {
                ChatMessage message = (ChatMessage) in.readObject();
                if (message == null) break;
                System.out.println("Mesaj de la " + message.getSender() + ": " + message.getMessage());
                if (clientName == null) {
                    clientName = message.getSender();
                }
                ChatServer.broadcast(message);
            }
        } catch (IOException | ClassNotFoundException e) {
            System.out.println("Clientul " + clientName + " s-a deconectat.");
        }
    }
}

```

```

    } finally {
        try { socket.close(); } catch (IOException e) { }
        ChatServer.removeClient(this);
    }
}

public void sendMessage(ChatMessage message) {
    try {
        out.writeObject(message);
        out.flush();
    } catch (IOException e) {
        e.printStackTrace();
    }
}
}
}

```

3. Clientul (ChatClient)

Clientul este responsabil pentru interacțiunea directă cu utilizatorul. El se conectează la server și oferă o interfață grafică intuitivă pentru trimiterea și primirea mesajelor.

Responsabilități:

- Stabilirea unei conexiuni către server prin intermediul unui *Socket*.
- Crearea și gestionarea fluxurilor de obiecte pentru comunicare (*ObjectOutputStream* și *ObjectInputStream*).
- Realizarea unei interfețe grafice cu ajutorul Swing
- Implementarea evenimentelor (prin *ActionListener*) pentru a detecta apăsările de buton sau taste (Enter) și a declanșa trimiterea mesajului.
- Lansarea unui fir separat pentru a asculta în mod continuu mesajele primite de la server, menținând astfel interfața reactivă.

```

import javax.swing.*;
import java.awt.*;
import java.awt.event.*;
import java.io.*;
import java.net.*;

```

```

public class ChatClient {

    private String serverAddress = "localhost";

    private int port = 12345;

    private ObjectOutputStream out;

    private ObjectInputStream in;

    private Socket socket;

    private JFrame frame;

    private JTextArea textArea;

    private JTextField inputField;

    private JButton sendButton;

    private String clientName;

    public ChatClient(String clientName) {

        this.clientName = clientName;

        frame = new JFrame("Chat Client - " + clientName);

        textArea = new JTextArea(20, 50);

        textArea.setEditable(false);

        inputField = new JTextField(40);

        sendButton = new JButton("Send");

        JPanel panel = new JPanel();

        panel.add(inputField);

        panel.add(sendButton);

        frame.getContentPane().add(new JScrollPane(textArea), BorderLayout.CENTER);

        frame.getContentPane().add(panel, BorderLayout.SOUTH);

        frame.pack();

        frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

        sendButton.addActionListener(new ActionListener() {

            public void actionPerformed(ActionEvent e) {

                sendMessage();

            }

        });

        inputField.addActionListener(new ActionListener() {

            public void actionPerformed(ActionEvent e) {

```

```

        sendMessage();
    }
});
}

public void start() {
    try {
        socket = new Socket(serverAddress, port);
        out = new ObjectOutputStream(socket.getOutputStream());
        in = new ObjectInputStream(socket.getInputStream());
        frame.setVisible(true);
        new Thread(new Runnable() {
            public void run() {
                try {
                    while (true) {
                        ChatMessage message = (ChatMessage) in.readObject();
                        if (message == null) break;
                        textArea.append(message.getSender() + ": " + message.getMessage() + "\n");
                    }
                } catch (Exception ex) {
                    ex.printStackTrace();
                }
            }
        }).start();
    } catch (IOException e) {
        e.printStackTrace();
    }
}

private void sendMessage() {
    String text = inputField.getText();
    if (text.trim().isEmpty()) return;
    ChatMessage message = new ChatMessage(clientName, text);
    try {

```

```

        out.writeObject(message);

        out.flush();
    } catch (IOException e) {
        e.printStackTrace();
    }

    inputField.setText("");
}

public static void main(String[] args) {
    String name = JOptionPane.showInputDialog("Introdu numele tău:");

    if (name == null || name.trim().isEmpty()) {
        name = "Anonymous";
    }

    ChatClient client = new ChatClient(name);
    client.start();
}
}

```

Cerințe atinse

1. Fluxuri:

- Pe client și pe server se utilizează *ObjectOutputStream* și *ObjectInputStream* pentru trimiterea și recepționarea obiectelor de tip *ChatMessage*.
- Aceasta permite transformarea obiectelor în biți și reconstrucția lor la destinație.

2. Serializare:

- Clasa *ChatMessage* implementează interfața *Serializable*, ceea ce face posibilă serializarea obiectelor.

3. Socketuri:

- Serverul creează un *ServerSocket* pe portul 12345 pentru a asculta conexiunile, iar clienții se conectează folosind un *Socket* către adresa serverului.
- Această metodă permite comunicarea între aplicații, chiar și pe calculatoare diferite.

4. Fire:

- Fiecare client este gestionat printr-un fir dedicat (clasa *ClientHandler* extinde *Thread*) pe server, asigurând execuția concurentă a conexiunilor.

- Pe client, un fir separat este folosit pentru a asculta mesajele de la server, menținând astfel interfața grafică reactivă.

5. **Arhitectura Client-Server:**

- Se separă clar responsabilitățile: serverul centralizează logica de procesare și transmitere a mesajelor, iar clienții se ocupă de interfața utilizator.

6. **Interfețe grafice & evenimente:**

- Clientul dispune de o interfață grafică realizată cu Swing (folosind componente precum *JFrame*, *TextArea*, *TextField* și *Button*).
- Evenimentele sunt gestionate prin *ActionListener*, declanșând acțiuni de trimitere a mesajelor atunci când se apasă butonul „Send” sau se apasă tasta Enter.

7. **Programare concurentă (Monitoare):**

- Sincronizarea este implementată în metoda *broadcast* (folosind cuvântul cheie *synchronized*) pentru a proteja accesul la colecția de clienți.
- Colecția *clientHandlers* este o colecție sincronizată, ceea ce previne conflictele între fire la adăugarea sau eliminarea handlerilor.